

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

REPORT DI PROGETTO

Ingegneria del Software: Tecniche Avanzate – Report del Progetto

Mario Celzo	Matricola: NF22500193
Renato Mancino	Matricola: NF22500242
Alfonso Maria Ferrara	Matricola: NF22500210

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

Indice

1 Descrizione del Sistema **1**

1.1 Contesto Applicativo 1

1.2 Lo Strumento BiblioFlow 1

1.2.1 Analisi del Funzionamento 2

2 Struttura del Sistema **3**

2.1 Package Principali del Sistema 3

2.1.1 app (pagine) 3

2.1.2 app/api (route handler) 4

2.1.3 lib 5

2.1.4 components 7

2.1.5 accessibility e contexts 7

2.1.6 hooks e types 10

2.1.7 prisma 10

2.1.8 public (PWA) 10

3 Flusso di Funzionamento del Sistema **12**

3.1 Tipologie di Utilizzo 12

3.2 Flusso di Utilizzo 12

3.2.1 Flusso di Utilizzo da parte dello Studente 12

3.2.2 Flusso di Utilizzo da parte dello Staff 13

3.2.3 Flusso delle Automazioni Pianificate 14

Elenco delle figure

2.1 Analisi modulo app (pagine). 4
2.2 Analisi modulo app/api. 6
2.3 Analisi modulo lib. 8
2.4 Analisi modulo components. 9
2.5 Analisi modulo prisma (modello dei dati). 11

3.1 Sequence Diagram della prenotazione. 13
3.2 Sequence Diagram del check-in. 14
3.3 Sequence Diagram dello staff. 15
3.4 Sequence Diagram delle automazioni pianificate. 15

CAPITOLO 1

Descrizione del Sistema

1.1 Contesto Applicativo

Nelle biblioteche universitarie i posti studio sono una risorsa limitata e molto contesa, soprattutto nei periodi d'esame. In assenza di un sistema di prenotazione l'accesso segue la regola del primo arrivato: chi abita vicino alla sede occupa i posti nelle prime ore del mattino, mentre gli studenti pendolari, che dipendono dagli orari dei mezzi pubblici, arrivano quando la sala è già piena. È diffusa anche l'occupazione «informale» del posto, ottenuta lasciando libri o zaino per ore, che rende la disponibilità reale diversa da quella apparente.

Il personale della biblioteca, d'altra parte, non dispone di dati affidabili sull'occupazione delle sale, sui posti che restano vuoti pur essendo stati promessi a qualcuno e sugli studenti che si prenotano senza presentarsi (*no-show*). Infine, gli studenti con esigenze di accessibilità hanno bisogno di sapere in anticipo quali posti sono raggiungibili e attrezzati.

La digitalizzazione della prenotazione dei posti, collegata a una conferma della presenza all'arrivo, permette di rendere l'accesso più equo e di liberare automaticamente i posti non utilizzati.

1.2 Lo Strumento BiblioFlow

BiblioFlow nasce in questo contesto come piattaforma web per la gestione dei posti studio e dei servizi di una biblioteca universitaria. È un'applicazione web progressiva (PWA) sviluppata con Next.js, React e TypeScript: si usa dal browser e può essere installata su smartphone. I dati sono conservati in PostgreSQL e gestiti tramite l'ORM Prisma; l'autenticazione è affidata ad Auth.js (NextAuth), con accesso tramite email e password o account Google.

Il sistema distingue tre ruoli:

- **Studente:** prenota un posto scegliendolo sulla mappa della sala, conferma la presenza con il check-in, consulta il catalogo, chiede libri in prestito o la loro preparazione al banco e riceve notifiche;
- **Bibliotecario:** gestisce prenotazioni, posti, prestiti, richieste e anomalie, registra il check-in degli studenti con lo scanner dei QR code o manualmente dall'elenco delle prenotazioni e consulta le statistiche;
- **Amministratore:** ha gli stessi strumenti del bibliotecario e in più può attivare o disattivare gli account.

Due caratteristiche distinguono BiblioFlow da un semplice calendario di prenotazione: l'opzione **pendolare**, con cui chi viaggia con i mezzi pubblici chiede un margine di tolleranza sull'orario di arrivo (nella versione di partenza il margine viene registrato nella prenotazione, ma né il check-in né il rilascio automatico dei posti lo applicano), e l'attenzione

all'**accessibilità**, con posti accessibili segnalati sulla mappa, preferenze di visualizzazione (alto contrasto, testo grande, movimento ridotto) e navigazione completa da tastiera.

1.2.1 Analisi del Funzionamento

Il funzionamento di BiblioFlow ruota attorno al ciclo di vita della prenotazione di un posto e può essere descritto attraverso i seguenti passaggi:

- **Autenticazione:** lo studente accede con email e password (verificata con bcrypt) o con Google. La sessione è un token JWT conservato in un cookie; un middleware intercetta le richieste e respinge quelle prive del cookie di sessione, mentre i controlli sul ruolo sono svolti dalle singole route e dal layout dell'area `/admin`.
- **Prenotazione guidata:** una procedura in quattro passi raccoglie data, durata (due ore, mezza giornata di mattina o di pomeriggio, giornata intera), sala e posto. La scelta del posto avviene su una mappa interattiva che mostra lo stato di ogni posto e le sue caratteristiche (presa elettrica, accessibilità). Il server verifica l'assenza di sovrapposizioni e gli orari di apertura della sala prima di creare la prenotazione nello stato `CONFERMATA`.
- **Check-in e uso del posto:** all'arrivo lo studente conferma la presenza dalla home, dove la prenotazione attiva è accompagnata da un QR code, oppure dalla pagina `/prenotazioni`; lo staff può registrarla con lo scanner o manualmente. Il check-in porta la prenotazione in `CHECK_IN` e il posto in `OCCUPATO`; il check-out, che lo studente esegue dalla pagina `/prenotazioni`, la porta in `COMPLETATA` e libera il posto.
- **Automazioni:** un'attività pianificata invia i promemoria di check-in, avvisa delle scadenze dei prestiti e rilascia i posti delle prenotazioni ancora prive di check-in quindici minuti dopo l'inizio (stato `NO_SHOW`).
- **Notifiche e tracciabilità:** le operazioni principali (creazione ed estensione delle prenotazioni, prestiti, interventi dello staff, automazioni) producono una notifica persistente per l'utente e un evento nel registro `LogEvento`, che alimenta la dashboard e la pagina delle anomalie dello staff. Fanno eccezione il check-in dalla home, che non registra nulla, e la cancellazione dalla pagina `/prenotazioni`, che elimina la prenotazione insieme ai suoi eventi.

CAPITOLO 2

Struttura del Sistema

L'architettura di BiblioFlow segue la struttura di un'applicazione Next.js con App Router: pagine e route handler convivono nella cartella `src/app`, mentre la logica condivisa, i componenti dell'interfaccia e il modello dei dati sono organizzati in package distinti, ognuno preposto a una specifica responsabilità. Di seguito viene presentata l'analisi dei moduli principali, così come appaiono nella versione di partenza del progetto. Completano il progetto la configurazione del monitoraggio degli errori con Sentry (`instrumentation.ts`, `sentry.*.config.ts`) e lo script `scripts/generate-secrets.js`, che genera i segreti richiesti dalle variabili d'ambiente.

2.1 Package Principali del Sistema

2.1.1 app (pagine)

L'obiettivo è fornire le schermate dell'applicazione. Ogni cartella sotto `src/app` corrisponde a un percorso dell'interfaccia; le pagine dello studente sono componenti client che dialogano con le API tramite `fetch`, mentre la dashboard e quasi tutte le pagine dello staff sono rese lato server e leggono i dati direttamente con Prisma (fanno eccezione richieste, scanner e statistiche, che usano le API `admin/*`).

Moduli principali:

- `layout.tsx`: struttura comune a tutte le pagine; monta i provider di tema, sessione, accessibilità e PWA, il collegamento per saltare al contenuto, le scorciatoie da tastiera, l'indicatore offline e il banner di installazione; l'intestazione è inclusa dalle singole pagine.
- `page.tsx`: home dello studente; mostra la prenotazione attiva più vicina con il QR code per il check-in e le scorciatoie verso le funzioni principali.
- `login`, `registrazione`, `recupera-password`, `reset-password`: accesso e gestione delle credenziali.
- `prenota`: procedura guidata di prenotazione in quattro passi (data, durata, sala, posto) con mappa interattiva e opzione pendolare.
- `prenotazioni` e `prenotazioni/[id]/estendi`: elenco delle prenotazioni, check-in, check-out, cancellazione ed estensione dell'orario.
- `libri`, `libri/[id]`, `prestiti`: catalogo, dettaglio di un libro, richiesta e rinnovo dei prestiti.
- `notifiche`, `profilo`: centro notifiche e dati del profilo, comprese le preferenze di pendolarismo e accessibilità.

- `admin/*`: area dello staff, con dashboard, gestione di prenotazioni, posti, prestiti, richieste di preparazione, utenti e anomalie, scanner dei QR code e statistiche.

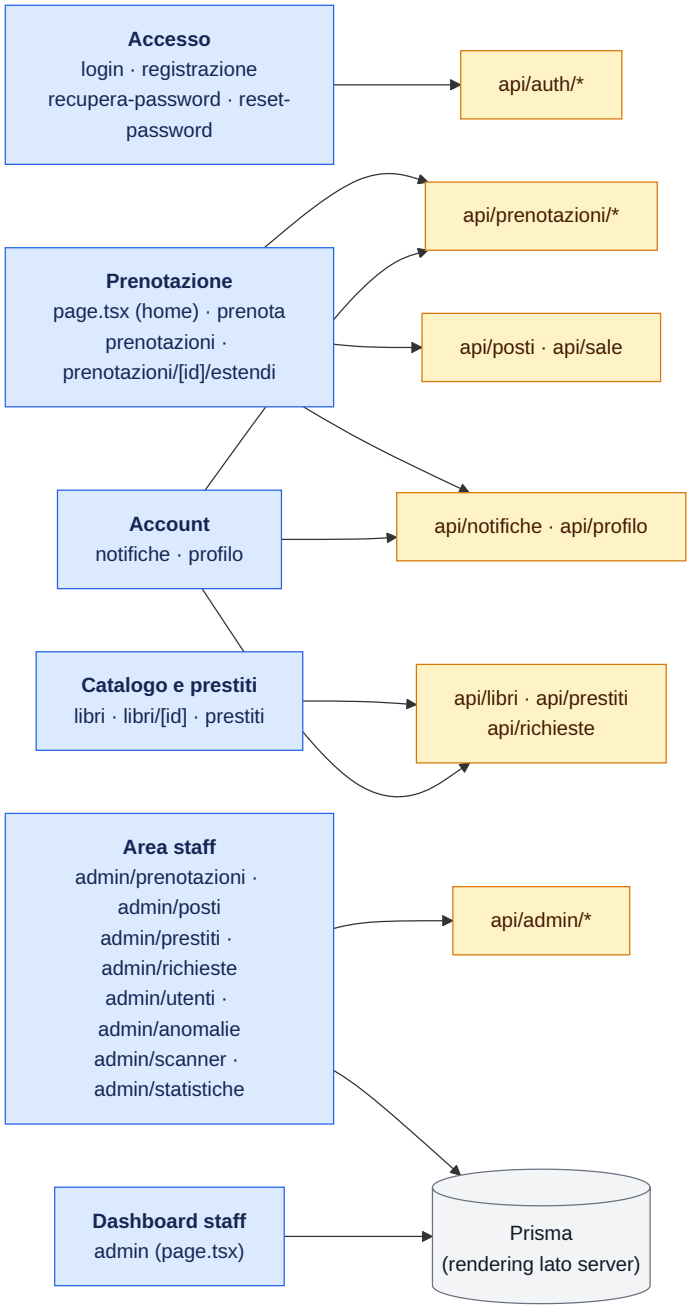


Figura 2.1: Analisi modulo app (pagine).

2.1.2 app/api (route handler)

Questo package rappresenta il backend dell'applicazione. Ogni file `route.ts` espone uno o più metodi HTTP e restituisce in genere JSON (`sse/posti` apre invece uno stream di eventi); le route validano l'input, verificano la sessione dove previsto e accedono ai dati tramite Prisma.

Moduli principali:

- `auth/*`: gestore di Auth.js e flussi di registrazione, verifica dell'email, recupero e reimpostazione della password; la registrazione e la richiesta di recupero sono soggette a limitazione della frequenza, l'accesso no.
- `prenotazioni`, `prenotazioni/[id]`, `prenotazioni/[id]/check-in`, `prenotazioni/[id]/estendi`: elenco, creazione, lettura, check-in, check-out, cancellazione ed estensione delle prenotazioni.
- `posti`, `sale`, `disponibilita-giorni`: stato dei posti, sale con i relativi orari e giorni prenotabili.
- `libri`, `prestiti`, `richieste`: catalogo, prestiti e rinnovi, richieste di preparazione dei libri al banco.
- `notifiche`, `profilo`: lettura, marcatura ed eliminazione delle notifiche; lettura e modifica del profilo.
- `admin/*`: operazioni dello staff (annullamento e modifica delle prenotazioni, stato dei posti, prestiti, richieste, utenti, anomalie, statistiche, validazione dei QR code).
- `cron/automations`: punto di ingresso dell'attività pianificata, protetto da un segreto condiviso.
- `sse/posti`: canale Server-Sent Events previsto per gli aggiornamenti in tempo reale dei posti.

A monte delle route opera `src/middleware.ts`, che lascia passare le pagine pubbliche (home `/`, `/login`, `/registrazione`, `/recupera-password`) e le API sotto `/api/auth` e `/api/cron`; per tutte le altre verifica soltanto la presenza del cookie di sessione: in sua assenza risponde 401 alle API e reindirizza le pagine al login, conservando la destinazione richiesta. La route `api/admin/richieste` non esegue alcun controllo di sessione o di ruolo.

2.1.3 lib

Raccoglie i servizi condivisi dal backend: autenticazione, accesso ai dati, automazioni e utilità trasversali.

Moduli principali:

- `auth.ts`: configurazione di Auth.js con i provider Credentials e Google, i callback che arricchiscono il token con ruolo e preferenze dell'utente e le funzioni `hasRole` e `isStaff`.
- `prisma.ts`: istanza unica del client Prisma collegata a PostgreSQL tramite l'adattatore `pg`.
- `automation-service.ts`: logica delle automazioni periodiche (`sendCheckInReminders`, `sendLoanExpiryAlerts`, `releaseNoShowReservations`) e l'orchestratore `runAllAutomations`, che le esegue in sequenza; esporta anche `notifyAvailableSeat`, pensata per avvisare gli utenti quando un posto si libera, che nella versione di partenza non è richiamata.
- `realtime-events.ts` e `sse-emitter.ts`: emettitore di eventi in memoria e funzioni di pubblicazione degli aggiornamenti dei posti e delle notifiche sul canale SSE; nella

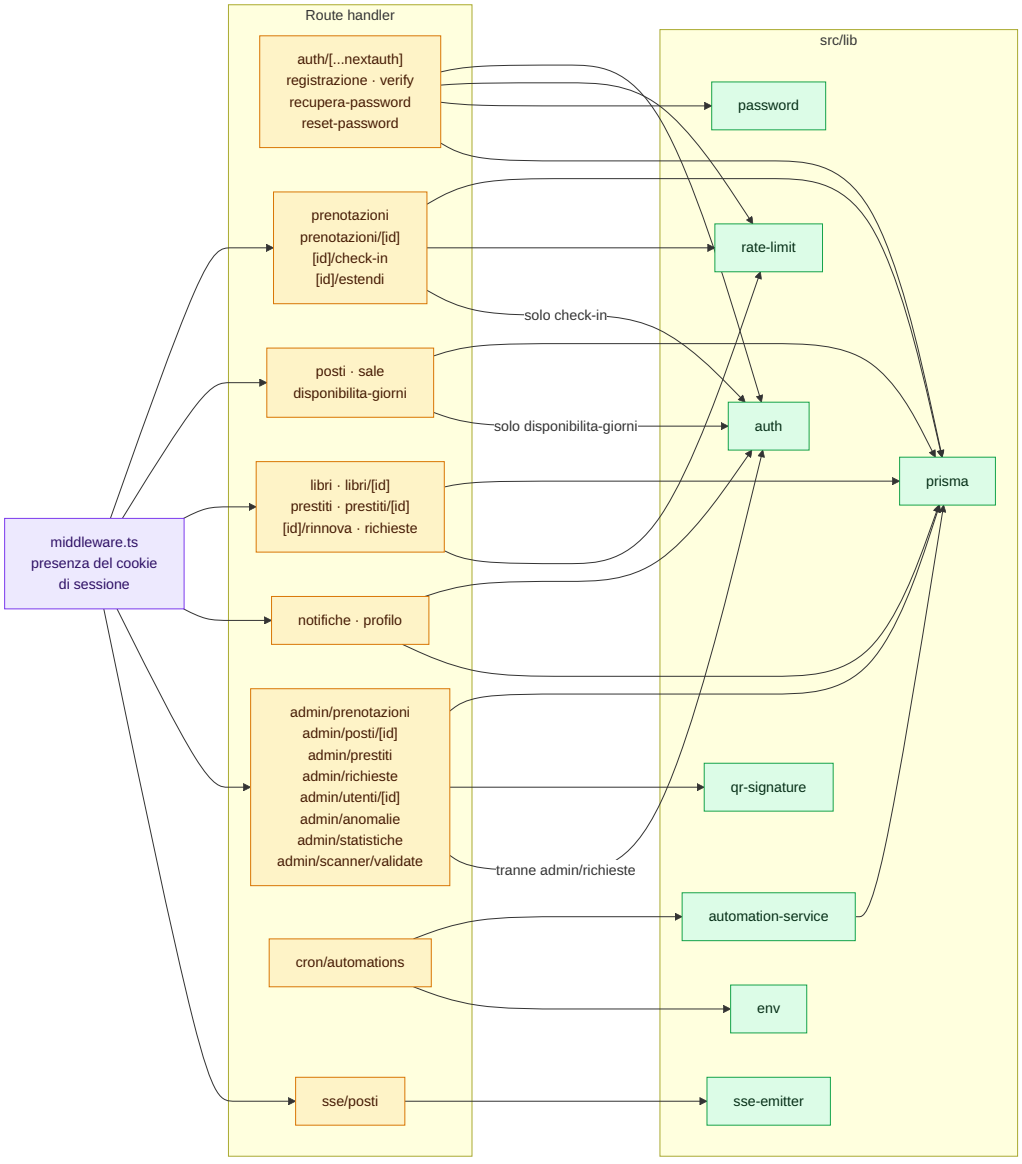


Figura 2.2: Analisi modulo app/api.

versione di partenza le funzioni di `realtime-events.ts` non sono richiamate da alcuna route, quindi il canale non riceve aggiornamenti.

- `qr-signature.ts`: firma HMAC-SHA256 dei QR code (`signQRCode`, `generateSignedQR`) e verifica di emittente, firma e scadenza a quindici minuti (`validateScannedQR`); nella versione di partenza è usata solo la verifica, dallo scanner, perché nessuna pagina genera QR firmati.
- `rate-limit.ts`: limitatori di frequenza in memoria per indirizzo IP e percorso; sono applicati a registrazione, recupero della password, prestiti e lettura dell'elenco delle prenotazioni, mentre quelli per l'accesso e per la creazione delle prenotazioni sono definiti ma non usati.
- `password.ts`, `env.ts`, `utils.ts`: hashing e regole delle password, validazione delle variabili d'ambiente all'avvio, composizione delle classi CSS.

2.1.4 components

Contiene i componenti React riutilizzabili che compongono l'interfaccia. Questo approccio modulare facilita la manutenzione e la coerenza visiva tra l'area dello studente e quella dello staff.

Moduli principali:

- `mappa-biblioteca.tsx`: mappa interattiva dei posti di una sala, con legenda degli stati e delle caratteristiche; sotto i 768 pixel di larghezza cede il posto a `mobile-posti-grid.tsx`, una griglia pensata per lo schermo dello smartphone.
- `qrcode-checkin.tsx`: mostra il QR code della prenotazione (tipo, identificativo e istante di generazione, senza firma) e invia la conferma di check-in.
- `admin/*`: menu di azioni su prenotazioni, posti, prestiti, utenti e anomalie, filtri, card della dashboard e grafici delle statistiche.
- `layout/*` e `providers/*`: intestazione con il contatore delle notifiche, barra laterale dello staff, provider di sessione, tema e PWA.
- `ui/*`: primitive grafiche (pulsanti, card, dialog, menu, schede, selettori) costruite su Radix UI.

2.1.5 accessibility e contexts

Questo package raccoglie il supporto all'accessibilità, che nel progetto è un requisito e non un'aggiunta.

Moduli principali:

- `contexts/accessibility-context.tsx`: stato globale delle preferenze (modalità accessibile, alto contrasto, testo grande, dimensione del carattere, movimento ridotto, supporto ai lettori di schermo, tema scuro, navigazione da tastiera), lette dal profilo dell'utente tramite `GET/api/profilo` e applicate all'intera interfaccia come classi del documento.
- `components/accessibility/*`: collegamento per saltare al contenuto e scorciatoie da tastiera, montati nel layout, più due componenti predisposti ma non ancora usati:

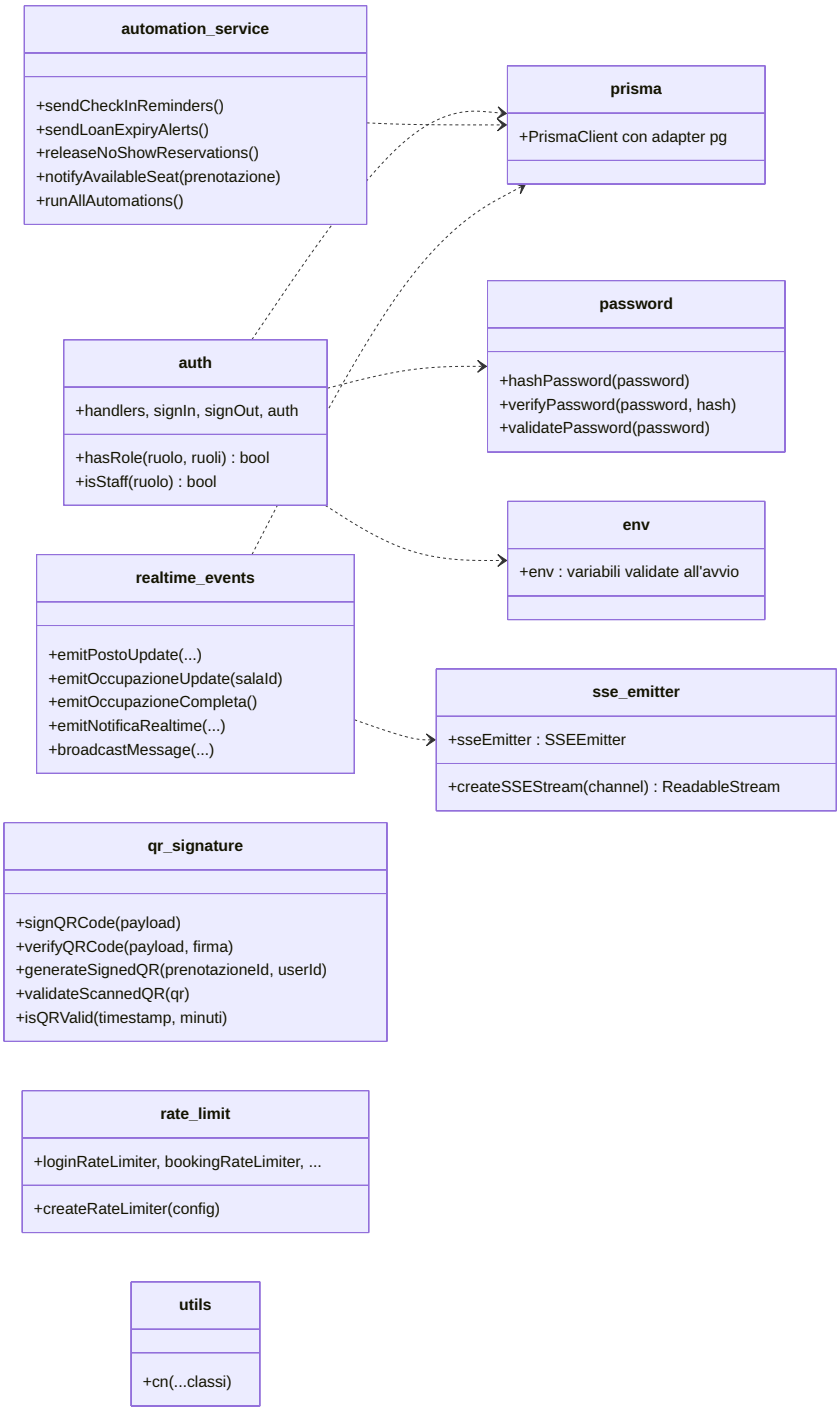


Figura 2.3: Analisi modulo lib.

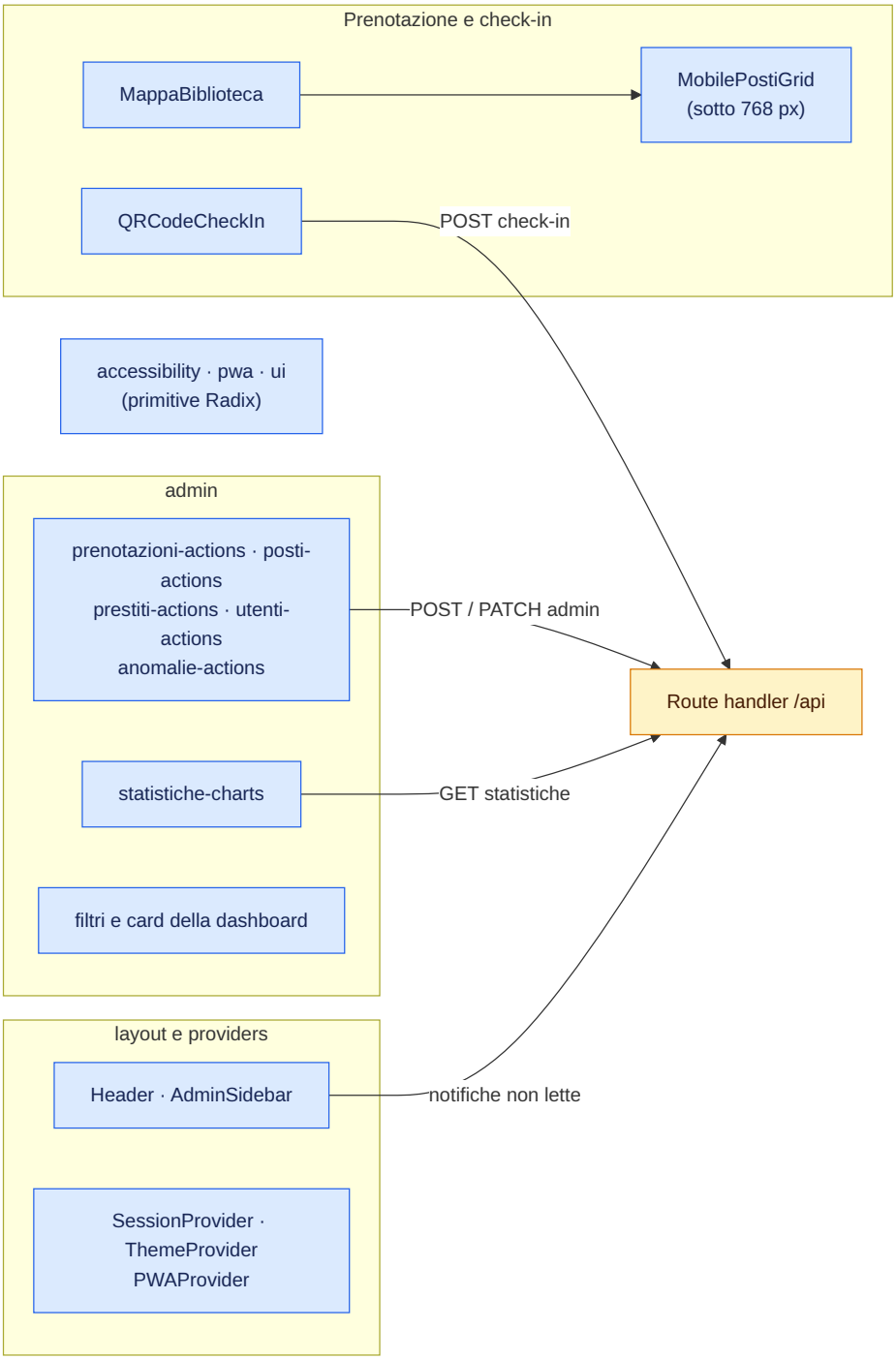


Figura 2.4: Analisi modulo components.

`FocusTrap`, per trattenere il focus nei dialog, e `LiveAnnouncer`, per gli annunci ai lettori di schermo.

2.1.6 hooks e types

Raccoglie il codice di supporto condiviso dal frontend: hook React e tipi del dominio.

Moduli principali:

- `hooks/use-sse.ts`: hook React che apre una connessione `EventSource`, gestisce la riconnessione e consegna gli eventi ricevuti; `usePostiRealtime` lo specializza per il canale dei posti. Nella versione di partenza nessuna pagina usa questi hook.
- `types/index.ts`: tipi TypeScript del dominio, predisposti per pagine e componenti ma non ancora importati.

2.1.7 prisma

Questo package contiene il modello dei dati e gli strumenti per crearlo.

Moduli principali:

- `schema.prisma`: definisce undici entità, tra cui utenti, sale, posti, prenotazioni, libri, prestiti, notifiche e registro degli eventi, e otto enumerazioni: ruoli, stati di posti, prenotazioni, prestiti e richieste di preparazione, tipi di notifica, di evento e di token. La data di una prenotazione è salvata come `@db.Date` e i suoi orari come `@db.Time`.
- `migrations/*`: storico delle migrazioni del database (due nella versione di partenza).
- `seed.ts`: svuota le tabelle principali e popola il database con dati dimostrativi: tre sale con i relativi posti, dieci libri, otto account (cinque studenti, due bibliotecari, un amministratore), prenotazioni, prestiti, notifiche ed eventi di esempio.

2.1.8 public (PWA)

Contiene le risorse statiche che rendono BiblioFlow installabile come applicazione.

Moduli principali:

- `manifest.json`: nome, icone, colori e scorciatoie dell'applicazione installata.
- `sw.js`: service worker con strategia *network first* per le pagine e le API e *cache first* per le risorse statiche; senza rete ripiega sulla copia in cache o su `/offline`, percorso che nella versione di partenza non esiste (la pagina statica è `offline.html`); inoltre l'elenco di precaricamento cita icone PNG assenti, per cui l'installazione del service worker non va a buon fine.
- `components/pwa/*`: banner di installazione e indicatore dello stato offline.

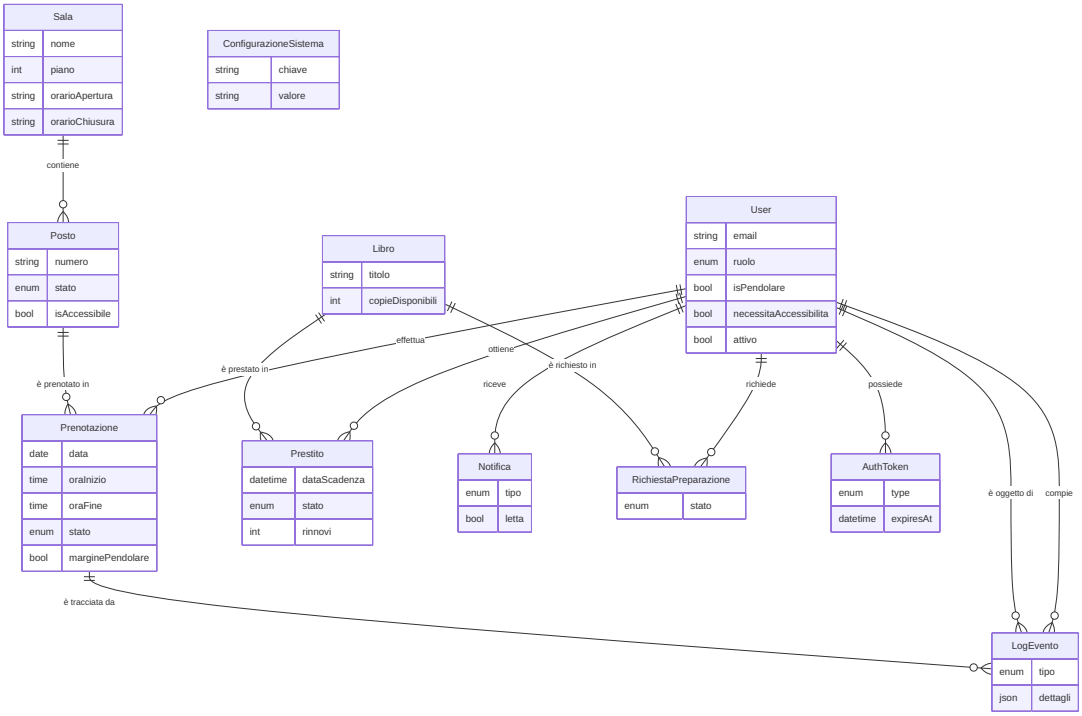


Figura 2.5: Analisi modulo prisma (modello dei dati).

CAPITOLO 3

Flusso di Funzionamento del Sistema

3.1 Tipologie di Utilizzo

BiblioFlow è stato progettato per accompagnare la giornata in biblioteca di chi studia e di chi gestisce le sale, affiancati da procedure automatiche eseguite una volta al giorno. Offre quindi tre modalità principali di utilizzo:

- **Utilizzo da parte dello studente:** tramite browser o applicazione installata, lo studente prenota il posto, riceve i promemoria e conferma la presenza con il check-in.
- **Utilizzo da parte dello staff:** bibliotecari e amministratori usano l'area `/admin` per registrare il check-in con lo scanner o manualmente, annullare o modificare prenotazioni e risolvere le anomalie.
- **Automazioni pianificate:** Vercel Cron invoca una volta al giorno, alle 9:00 UTC (pianificazione `0 9 * * *` in `vercel.json`), l'endpoint `/api/cron/automations`, che esegue promemoria, avvisi e rilascio dei posti non occupati.

3.2 Flusso di Utilizzo

In questa sezione vengono dettagliati i flussi operativi che caratterizzano le tre modalità di utilizzo. Per ognuna si descrive la sequenza di eventi che porta dall'azione dell'utente, o dell'attività pianificata, al risultato finale.

3.2.1 Flusso di Utilizzo da parte dello Studente

Prenotazione.

1. Lo studente apre `/prenota` e sceglie la data, la durata e la sala; la pagina carica sale e posti con `GET/api/sale` e `GET/api/posti`.
2. Sulla mappa sceglie un posto libero, eventualmente attiva il margine pendolare, e conferma.
3. La pagina invia `POST/api/prenotazioni` con identificativo dell'utente, posto, data e orari.
4. Il route handler controlla i campi obbligatori, legge utente, posto e sala, verifica che non esistano prenotazioni attive sovrapposte per l'utente o sul posto e che l'intervallo rientri nell'orario di apertura della sala.
5. Se i controlli sono superati crea la prenotazione nello stato `CONFERMATA`, registra l'evento `PRENOTAZIONE_CREATA`, invia una notifica e risponde 201; in caso di sovrapposizione risponde 409, con un orario fuori apertura 400.

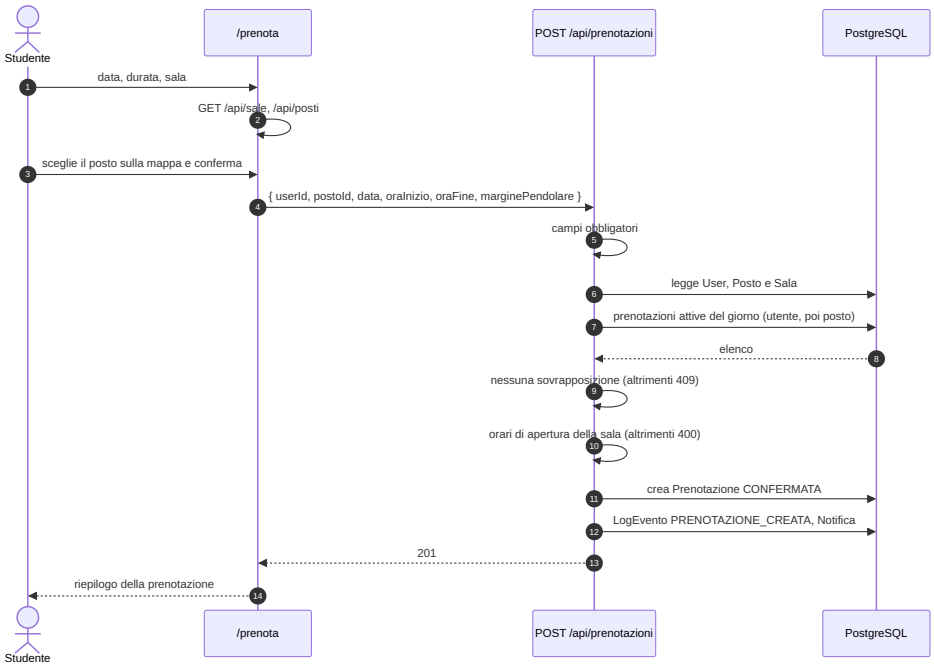


Figura 3.1: Sequence Diagram della prenotazione.

Check-in.

1. La home mostra la prenotazione attiva più vicina, di oggi o di un giorno successivo, con un pulsante che apre il QR code e la conferma della presenza.
2. La conferma invia `POST/api/prenotazioni/[id]/check-in`; il route handler verifica la sessione, legge la prenotazione e controlla che appartenga all'utente e sia `CONFERMATA`.
3. Se la richiesta cade nei quindici minuti che precedono l'inizio, la prenotazione passa a `CHECK_IN` e il posto a `OCCUPATO`; altrimenti la richiesta è rifiutata con 400 («troppo presto» oppure «periodo di check-in scaduto»).

Nota sul Sequence Diagram. Il route handler confronta l'istante della richiesta con `oraInizio` così come è salvata (`@db.Time`, cioè un orario del primo gennaio 1970) senza combinarla con la data della prenotazione: nella versione di partenza la richiesta risulta sempre successiva all'inizio e riceve sempre la risposta 400 «periodo di check-in scaduto». Il check-in dello studente riesce invece dalla pagina `/prenotazioni`, che usa `PATCH/api/prenotazioni/[id]` con azione `check-in` e non applica alcuna finestra temporale.

3.2.2 Flusso di Utilizzo da parte dello Staff

1. Il bibliotecario apre `/admin/scanner` e inquadra il QR code mostrato dallo studente.
2. La pagina invia il contenuto del QR a `POST/api/admin/scanner/validate`, che verifica sessione e ruolo, la firma e la validità temporale del codice.
3. Il route handler controlla che la prenotazione esista, appartenga allo studente indicato nel QR, non sia annullata, scaduta, già in check-in o completata e sia fissata per il giorno

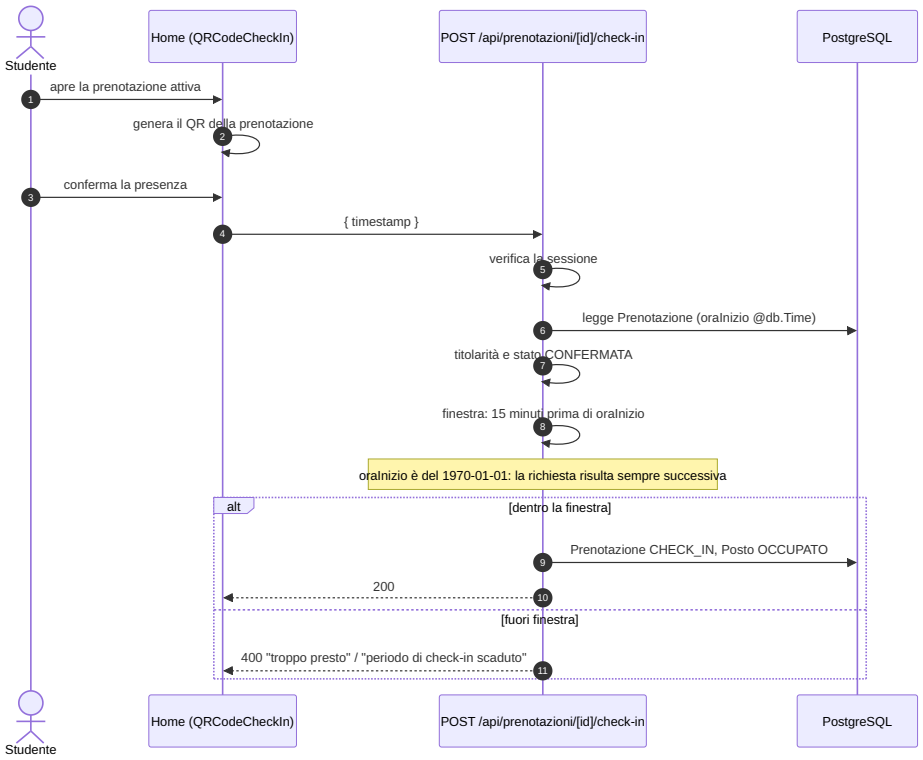


Figura 3.2: Sequence Diagram del check-in.

corrente; quindi registra il check-in, occupa il posto, crea l'evento `CHECK_IN` e la notifica e mostra allo staff i dati dello studente.

4. Dalla pagina `/admin/prenotazioni` lo staff può annullare una prenotazione (`POST/api/admin/prenotazioni`, azione `ANNULLA_SINGOLA`): la prenotazione diventa `CANCELLATA`, il posto torna disponibile se era occupato e lo studente riceve una notifica.

Nota sul Sequence Diagram. Il diagramma descrive la validazione di un QR firmato con `generateSignedQR`. Nella versione di partenza nessuna pagina richiama questa funzione: il QR mostrato dalla home contiene solo tipo, identificativo della prenotazione e istante di generazione, senza utente, firma ed emittente, e lo scanner lo rifiuta come «non è un QR BiblioFlow». Lo staff può comunque registrare il check-in da `/admin/prenotazioni` con l'azione `CHECK_IN_MANUALE` di `POST/api/admin/prenotazioni`.

3.2.3 Flusso delle Automazioni Pianificate

1. Ogni giorno alle 9:00 UTC Vercel Cron chiama `GET/api/cron/automations`, passando il segreto `CRON_SECRET` nell'intestazione di autorizzazione.
2. Il route handler confronta il token con il segreto configurato e invoca `runAllAutomations()`.
3. Il servizio invia i promemoria di check-in alle prenotazioni che iniziano tra quindici e venti minuti, gli avvisi per i prestiti che scadono tra tre giorni o il giorno successivo, e rilascia le prenotazioni ancora `CONFERMATA` (senza check-in) iniziate da più di quindici minuti: stato `NO_SHOW`, posto `DISPONIBILE`, notifica all'utente ed evento `NO_SHOW_AUTO`.

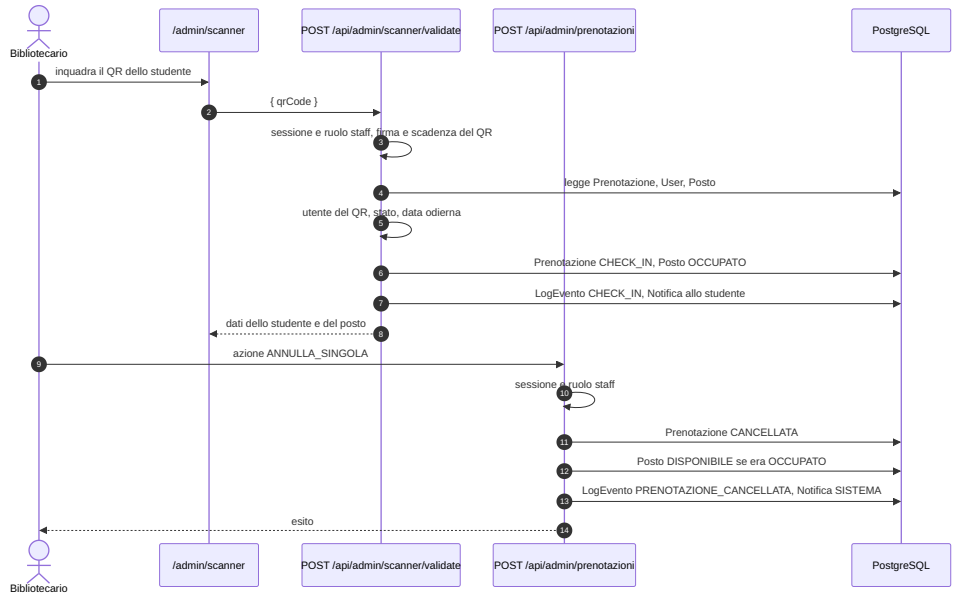


Figura 3.3: Sequence Diagram dello staff.

4. Al termine restituisce un riepilogo delle operazioni eseguite.

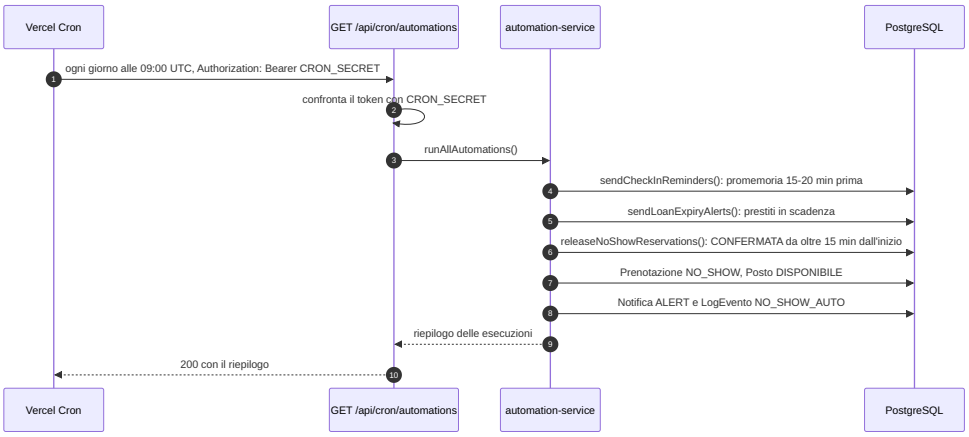


Figura 3.4: Sequence Diagram delle automazioni pianificate.

Nota sul Sequence Diagram. Poiché l'attività gira una sola volta al giorno, i promemoria raggiungono soltanto le prenotazioni che iniziano tra le 9:15 e le 9:20 UTC e il rilascio dei no-show avviene al più una volta al giorno, anche se il commento del route handler indica un'esecuzione ogni cinque minuti.